



La Sobremesa

Documentación técnica del sistema de gestión de restaurante

Stack	Python · Flask · SQLAlchemy · PostgreSQL · Bootstrap 5 · Chart.js
Arquitectura	MVC (Modelo–Vista–Controlador)
Repositorio	github.com/alcedoanggi-crypto/Restaurantes_mvc
Fecha	Septiembre 2026

1. Resumen del sistema

La **Sobremesa** es un sistema de gestión para restaurante que cubre el flujo completo de una comanda: toma de pedidos en mesa (POS), pantalla de cocina en tiempo real (KDS), cobro y facturación en caja, control de mesas, gestión de menú, inventario de insumos, administración de personal/usuarios y un dashboard con reportes y gráficos.

Está construido en **Python** con el micro-framework **Flask**, siguiendo una arquitectura **MVC** explícita: los *modelos* (SQLAlchemy) representan las tablas de la base de datos, los *controladores* (blueprints de Flask) contienen la lógica de cada módulo y las *vistas* (plantillas Jinja2 + Bootstrap 5) renderizan el HTML.

2. Arquitectura y organización del código

```
restaurante-mvc/
├─ run.py           # punto de entrada (flask run / python run.py)
├─ config.py        # configuración: SECRET_KEY, DATABASE_URL, IVA, umbrales KDS
├─ seed.py          # datos de demostración (usuarios, mesas, menú, insumos)
├─ app/
│   ├── __init__.py # application factory: create_app()
│   ├── extensions.py # instancias compartidas: db, login_manager, migrate
│   ├── security.py  # roles del sistema y decorador @roles_required
│   ├── models/      # MODELO – clases SQLAlchemy (una tabla por archivo)
│   ├── controllers/ # CONTROLADOR – blueprints (auth, pos, cocina, caja, ...)
│   ├── services/reportes.py # lógica de negocio de KPIs, series y cierre de caja
│   ├── templates/   # VISTA – Jinja2 (una carpeta por módulo)
│   └─ static/       # CSS (theme.css), JS (dashboard.js, kds.js), imágenes
```

2.1 Application factory (`app/__init__.py`)

`create_app()` construye la instancia de Flask, carga la configuración, inicializa las extensiones (`db` , `migrate` , `login_manager`) y registra los diez *blueprints* del sistema. También agrega manejadores de error (403/404), un filtro Jinja `money` para formatear montos y variables globales de plantilla (roles, fecha actual).

2.2 Extensiones (`app/extensions.py`)

Se definen una sola vez las instancias compartidas para evitar imports circulares (patrón *application factory*):

`SQLAlchemy()` , `Migrate()` y `LoginManager()` . El `user_loader` de Flask-Login recupera al usuario autenticado por `id` desde la tabla `usuarios` .

2.3 Seguridad y roles (`app/security.py`)

El sistema define cuatro roles: `administrador` , `mesero` , `cocina` y `cajero` . El decorador

`@roles_required(*roles)` se aplica a cada vista y verifica que el usuario esté autenticado y que su rol esté en la lista permitida (el rol `administrador` siempre tiene acceso). Si no cumple, responde `403` .

```
def roles_required(*roles):
    def decorator(view):
        @wraps(view)
        def wrapped(*args, **kwargs):
            if not current_user.is_authenticated:
                return redirect(url_for("auth.login"))
            if current_user.rol != ADMIN and current_user.rol not in roles:
                abort(403)
            return view(*args, **kwargs)
        return wrapped
    return decorator
```

2.4 Autenticación (`app/controllers/auth.py`)

El login acepta usuario o correo. La contraseña nunca se guarda en texto plano: se compara con

`check_password_hash` contra el `password_hash` generado con `werkzeug.security.generate_password_hash`

(PBKDF2). Flask-Login gestiona la sesión (`login_user` / `logout_user`) y redirige a la página originalmente solicitada (`?next=`) tras iniciar sesión.

3. Lógica de negocio por módulo

3.1 Ciclo de vida de un pedido

Un pedido recorre los siguientes estados, controlados desde distintos módulos según el rol:



Mesero (POS)	abre la mesa → crea <code>Pedido</code> en estado <code>abierto</code>
Mesero (POS)	agrega platillos → cada uno crea un <code>DetallePedido</code> en <code>pendiente</code>
Cocina (KDS)	ve la comanda en tiempo real (polling cada 5–7s) y avanza cada renglón: <code>pendiente</code> → <code>en_preparación</code> → <code>listo</code>
Sistema	<code>Pedido.recalcular_estado()</code> deriva el estado del pedido automáticamente a partir de sus renglones
Mesero (POS)	marca el pedido como <code>servido</code> al entregarlo en la mesa
Caja	calcula IVA, registra la <code>Factura</code> (folio, método de pago, cambio) y cobra
Sistema	el pedido pasa a <code>cobrado</code> y la mesa vuelve a <code>libre</code>

El método `Pedido.recalcular_estado()` deriva el estado del pedido a partir del estado de sus renglones (`DetallePedido`), sin necesidad de que el mesero lo actualice manualmente: si todos los renglones están `listo`, el pedido pasa a `listo`; si alguno sigue `pendiente` o `en_preparación`, el pedido queda `en_preparación`.

3.2 POS — Toma de pedidos (`controllers/pos.py`)

- Al entrar a una mesa (`/pos/mesa/<id>`) se reutiliza el pedido activo o se crea uno nuevo y la mesa pasa a `ocupada`.
- Al agregar un platillo, si ya existe un renglón idéntico **pendiente** (mismo platillo y notas), se suma la cantidad en vez de duplicar la fila.
- Solo se pueden eliminar renglones en estado `pendiente` (si ya pasaron a cocina, no).
- Cancelar un pedido solo es posible si ningún renglón está en preparación o listo.

3.3 Cocina / KDS (`controllers/cocina.py`)

Lista los pedidos activos ordenados por antigüedad. Cada renglón calcula sus `minutos_espera` (desde su creación hasta que se marca `listo`) y el ticket se colorea con un semáforo configurable (`KDS_WARN_MIN=8`, `KDS_LATE_MIN=15` en `config.py`): verde si está a tiempo, amarillo en advertencia, rojo si va tarde. La pantalla se refresca sola vía *polling* AJAX a `/api/cocina` cada 5–7 segundos (sin WebSockets).

3.4 Caja y facturación (`controllers/caja.py`)

Al cobrar un pedido se calcula `subtotal` , `impuesto` (IVA configurable) y `total` con redondeo `ROUND_HALF_UP` a centavos. Se genera un folio único con el formato `F-YYYYMMDD-NNNN` y se crea el registro `Factura` con método de pago, monto recibido y cambio. El pedido pasa a `cobrado` y la mesa vuelve a `libre` . El cierre diario (`/caja/cierre`) agrupa las facturas del día por método de pago.

Nota sobre "factura": en este sistema la *Factura* es un comprobante de venta interno (folio + IVA + total), equivalente a un ticket de caja. No es una factura fiscal electrónica timbrada ante una autoridad tributaria (no genera CFDI/NCF ni se conecta a un dispositivo de impresión fiscal certificado); esa integración quedaría pendiente y depende del país/normativa.

3.5 Dashboard y reportes (`services/reportes.py`)

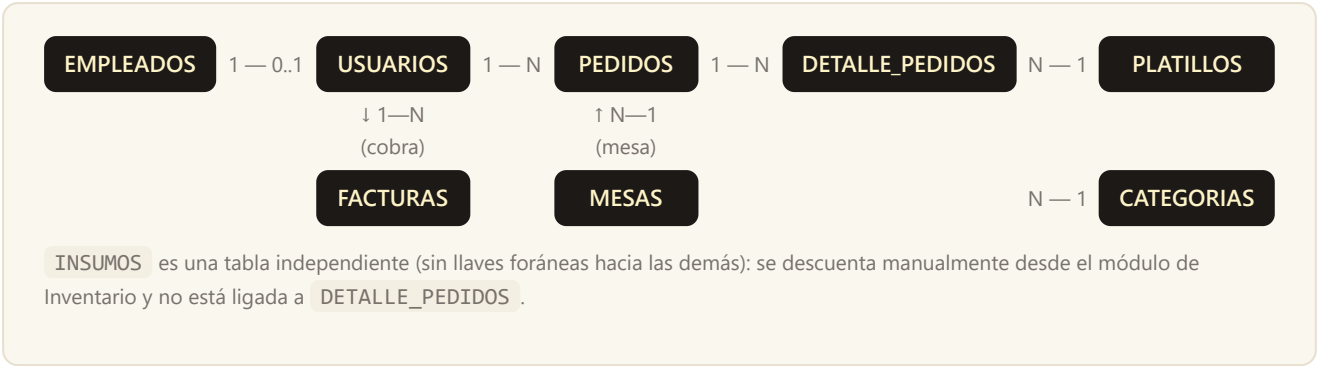
Toda la agregación de KPIs (ventas del día, ticket promedio, mesa más rentable, platillo estrella, series de ventas por día/mes, ocupación de mesas) se calcula con consultas SQLAlchemy (`func.sum` , `func.count` , joins) y se procesa en Python para evitar depender de funciones de fecha específicas de un motor de base de datos, permitiendo usar PostgreSQL o SQLite indistintamente. Los datos se exponen a Chart.js vía `/api/dashboard/charts` .

3.6 Inventario y menú

`Insumo` guarda stock actual/mínimo y costo unitario; expone las propiedades calculadas `bajo_stock` y `valor_inventario` . El menú organiza `Platillo` dentro de `Categoria` (con orden de despliegue) y permite alternar disponibilidad sin borrar el platillo, para no perder el historial de ventas asociado.

4. Base de datos

El ORM es **SQLAlchemy** a través de **Flask-SQLAlchemy**, con migraciones gestionadas por **Flask-Migrate** (Alembic). El motor principal usado en desarrollo es **PostgreSQL 17**; `config.py` normaliza la URL de conexión y fuerza el driver **psycopg v3** (`postgresql+psycopg://`), ya que `psycopg2` no compila en Python 3.13. Si no se define `DATABASE_URL`, la app cae automáticamente a un archivo **SQLite** local (`restaurante.db`), útil para pruebas rápidas sin instalar un servidor de base de datos.



4.1 Tablas y campos principales

Tabla	Campos clave	Propósito
usuarios	username, email, password_hash, rol, activo, empleado_id (FK)	Cuentas de acceso y su rol (administrador/mesero/cocina/cajero)
empleados	nombre, apellido, puesto, salario, fecha_contratacion	Ficha de personal, opcionalmente ligada a un usuario
mesas	numero, capacidad, ubicacion, estado (libre/ocupada/reservada)	Mesas físicas del restaurante
categorias	nombre, orden	Agrupación del menú (entradas, fuertes, postres, bebidas)
platillos	nombre, precio, categoria_id (FK), disponible, tiempo_preparacion	Catálogo de platillos vendibles
pedidos	mesa_id (FK), usuario_id (FK), estado, notas, creado_en	Una comanda abierta por un mesero en una mesa
detalle_pedidos	pedido_id (FK), platillo_id (FK), cantidad, precio_unitario, estado	Cada renglón/platillo dentro de un pedido, con su propio avance en cocina
facturas	folio, pedido_id (FK), cajero_id (FK), subtotal, impuesto, total, metodo_pago	Comprobante de cobro de un pedido
insumos	nombre, unidad, stock_actual, stock_minimo, costo_unitario	Inventario de materia prima

4.2 Migraciones

```
flask --app run db init
flask --app run db migrate -m "esquema inicial"
flask --app run db upgrade
```

5. Puesta en marcha

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt

copy .env.example .env
python seed.py          # crea las tablas y carga datos demo
python run.py           # http://127.0.0.1:5000
```

6. Credenciales

Uso exclusivo de desarrollo/demo. Estas credenciales están pensadas para el entorno local del proyecto (ya incluidas como valores de ejemplo en `.env.example` dentro del repositorio). Antes de exponer el sistema en un entorno real, deben cambiarse `SECRET_KEY`, la contraseña de la base de datos y las contraseñas de todos los usuarios.

6.1 Usuarios de la aplicación (login)

Administrador	admin	admin123
Mesero	mesero	mesero123
Cocina	cocina	cocina123
Cajero	cajero	cajero123

6.2 Conexión a la base de datos (PostgreSQL local)

Host	localhost:5432
Base de datos	restaurante
Usuario	restaurante
Contraseña	123456
URL completa	postgresql://restaurante:123456@localhost:5432/restaurante

Definidas en `.env` (ignorado por git) a partir de la plantilla `.env.example`. `config.py` convierte automáticamente el prefijo a `postgresql+psycpg://` para usar el driver `psycpg` v3.

Documento generado a partir del código fuente del repositorio La Sobremesa.